

Análisis de Diseño y Complejidad: Hoja de Cálculo Dispersa

Integrantes: Fátima Villón, Andrea Loyola, Leonardo Chocce

1. Fundamentación de la Arquitectura de Datos

El diseño se basa en una **Matriz Dispersa** mediante **Listas Enlazadas Cruzadas**. Esta arquitectura es óptima para hojas de cálculo con baja densidad de datos (*sparse data*), evitando el desperdicio de memoria de las matrices densas convencionales.

Hitos técnicos de la implementación:

- **Escalabilidad Dinámica:** El método `insert` expande los vectores de cabeceras mediante `resize` al detectar índices fuera de rango, permitiendo un crecimiento asintótico de la hoja.
- **Eficiencia Espacial:** Solo se instancian nodos para celdas con contenido explícito. Cada nodo emplea punteros ortogonales (`next_row`, `next_col`) para mantener la coherencia estructural.
- **Interfaz Responsiva:** Se optimizó el renderizado mediante la captura de eventos `Resized` en SFML, recalculando el *viewport* para procesar solo las celdas visibles en pantalla.

2. Análisis de Complejidad de las Operaciones Implementadas

Se define k como el número de elementos instanciados en una fila o columna y R como la cantidad de filas afectadas en un rango.

Operación	Complejidad	Análisis Técnico
<code>insert / modify</code>	$O(k)$	Localización lineal de la coordenada en la lista enlazada para el correcto direccionamiento de punteros.
<code>get</code> (Consulta)	$O(k)$	Recorrido secuencial sobre los nodos activos en la fila o columna hasta hallar la intersección de índices.
<code>remove</code> (Celda)	$O(k)$	Búsqueda del nodo y reasignación de punteros de los nodos adyacentes para preservar la integridad de la malla.
<code>deleteRow / Col</code>	$O(k \cdot m)$	Eliminación iterativa de los m nodos presentes en la línea, ajustando los enlaces ortogonales de las dimensiones opuestas.
<code>sum / average</code>	$O(k \cdot R)$	Procesamiento exclusivo de nodos numéricos instanciados dentro del rango, optimizando el cálculo al ignorar celdas nulas.
<code>max / min</code>	$O(k \cdot R)$	Comparación iterativa sobre los nodos existentes en el área rectangular definida por el usuario.

3. Conclusiones sobre el Diseño de Software

La implementación de un **intérprete de comandos** (`executeCommand`) y la función de navegación rápida (`/GOTO`) optimizan la usabilidad profesional del software. El desarrollo manual de la lógica de punteros, sin recurrir a contenedores asociativos de la STL, garantiza un control absoluto sobre el ciclo de vida de los objetos en memoria y una comprensión profunda de la eficiencia algorítmica en C++.